LLM Hallucination Detection via Uncertainty

SEDS 537 Machine Learning Term Project

May 26, 2026

Abstract

Large language models can generate fluent answers that are factually incorrect or unsupported by evidence. This project studies hallucination detection as a binary classification problem, where a candidate answer is classified as supported/truthful or hallucinated/unsupported. Instead of asking a language model to directly judge hallucination, the proposed approach uses open-source language models as uncertainty feature extractors. Qwen2.5-3B and Qwen2.5-0.5B are used to score answer tokens, and features such as token probability, log-probability, low-confidence ratio, and entropy are computed. These features are then used to train Logistic Regression, Linear SVM, RBF SVM, and Random Forest classifiers.

Experiments are conducted on HaluEval QA and TruthfulQA. HaluEval is evaluated both with its supporting context and after removing the context, while TruthfulQA is used as a no-context truthfulness dataset. Results show that uncertainty features are highly effective on context-grounded HaluEval, but performance decreases when context is removed and drops further on TruthfulQA. Grouped cross-validation, model-size comparison, and ablation experiments show that combining probability, log-probability, and entropy features gives the strongest same-dataset performance. The findings suggest that uncertainty-based hallucination detection is promising for context-grounded settings, but no-context truthfulness detection remains more challenging due to dataset shift, common misconceptions, and the lack of external evidence.

1 Introduction

Large language models (LLMs) are able to generate fluent and coherent text across a wide range of tasks, including question answering, summarization, and dialogue. However, fluency does not guarantee factual correctness. A model may produce an answer that sounds plausible but is factually incorrect, unsupported by the given evidence, or inconsistent with known information. This behavior is commonly referred to as hallucination. Hallucinations are especially difficult to detect because the generated text can appear natural and confident even when the underlying content is wrong.

Detecting hallucinations is important because LLM outputs are increasingly used in applications where reliability matters. In educational, professional, medical, legal, and information-seeking settings, an unsupported answer can mislead users even if it is written fluently. A hallucination detector can act as an additional reliability layer by estimating whether a generated answer should be trusted, flagged, or checked against external evidence. This is particularly useful when LLM systems are used for factual question answering or decision-support tasks, where incorrect information can reduce user trust and lead to poor downstream decisions.

One possible approach is to ask an LLM directly whether an answer is hallucinated. However, this creates a black-box judgment that can be sensitive to prompting, model knowledge, and instruction-following behavior. In contrast, uncertainty-based detection treats the LLM as a feature extractor rather than as the final judge. The model's token probabilities and entropy values provide numerical signals that can be compared, analyzed, and evaluated with standard machine learning methods. This makes it possible to study which signals are useful, whether results are stable across train-test splits, and how performance changes across datasets.

This project formulates hallucination detection as a binary classification problem. Given a question, a candidate answer, and optionally a context passage, the goal is to predict whether the answer is supported or hallucinated. The answer is scored token by token using open-source Qwen models, and numerical features such as token probability, log-probability, low-confidence token ratio, and entropy are extracted. These features are then used to train classical machine learning classifiers, including Logistic Regression, Linear SVM, RBF SVM, and Random Forest. The use of classical classifiers also makes the final decision process easier to evaluate through cross-validation, ablation studies, confusion matrices, and ROC analysis.

The experimental evaluation uses HaluEval QA and TruthfulQA. HaluEval provides question-answer examples with supporting context, making it suitable for context-grounded hallucination detection. TruthfulQA does not provide context and instead tests whether models can distinguish truthful answers from common misconceptions. This distinction is important because uncertainty features are influenced by the input used during scoring. If the feature extractor sees a context passage, its token probabilities may reflect agreement or disagreement with that evidence. Without context, the same feature extractor must rely more heavily on its internal knowledge and calibration.

To better understand the role of context, this project also creates a no-context version of HaluEval by removing the context passage before feature extraction. This allows the experiments to compare three settings: HaluEval with context, HaluEval without context, and TruthfulQA without context. The main objective is therefore not only to measure classification performance, but also to understand when uncertainty features work well and where they fail. This distinction helps avoid overclaiming the results: strong performance on context-grounded examples does not necessarily imply equally strong open-domain truthfulness detection.

2 Related Work

This project is closely connected to two benchmark datasets: HaluEval and TruthfulQA. HaluEval was introduced as a large-scale benchmark for evaluating hallucination in LLM outputs across several task types, including question answering, dialogue, summarization, and general user queries [7]. It is especially useful for this project because its QA split provides a candidate answer, a supporting context passage, and a hallucination label. This makes it possible to study hallucination detection as a supervised binary classification problem. TruthfulQA measures a different but related problem: whether models imitate common human falsehoods and misconceptions [8]. Unlike HaluEval QA, TruthfulQA does not provide a supporting context passage, which makes it useful for testing whether a detector trained on uncertainty features can generalize to no-context truthfulness evaluation.

Several prior studies motivate the use of token-level uncertainty for hallucination detection. The stronger-focus approach shows that uncertainty-based detection can be improved by paying attention to more informative tokens instead of treating every token equally [3]. This idea supports the use of token probability, log-probability, and low-confidence token ratios in this project. A related HaluEval-based calibration study also connects hallucination detection with confidence calibration and shows that standard classifiers and calibration metrics can be useful when evaluating hallucination detectors [1]. These works are important because they treat model confidence as a measurable signal rather than relying only on direct LLM self-judgment.

Entropy-based methods are another important direction in the literature. Semantic entropy estimates uncertainty over meanings rather than only over surface-level token choices [4]. This is important because two different generated answers may use different words while expressing the same meaning. Semantic Entropy Probes extend this idea by learning cheaper probe-based approximations of semantic uncertainty, reducing the cost of repeated generation [6]. Compared with these methods, this project uses a simpler and more practical single-pass feature extraction setup. However, the entropy features used here are motivated by the same general idea: uncertain generations should show stronger probability spread or weaker confidence signals.

Other work studies hallucination detection through reasoning, polling, or probabilistic belief modeling. ChainPoll uses repeated LLM judgments to detect hallucinations and emphasizes that correctness and adherence to the requested task are both important for evaluation [5]. Belief Tree Propagation proposes a probabilistic framework that reasons over related claims to reduce the effect of noisy individual judgments [2]. These methods are more complex than the approach used in this project, but they show why a single score or one direct model judgment may be unreliable. They also motivate future extensions where uncertainty features could be combined with consistency or evidence-based signals.

The reviewed literature also shows an important difference between detecting unsupported answers with evidence and detecting false answers without evidence. In context-grounded benchmarks such as HaluEval QA, the detector can use the relationship between the context passage and the candidate answer. In no-context benchmarks such as TruthfulQA, the model must rely more on prior knowledge and calibration. This distinction is central to the experiments in this project.

Overall, prior work suggests that hallucination detection can be approached through benchmark construction, token uncertainty, semantic uncertainty, calibration, self-consistency, and evidence-based reasoning. This project focuses on the uncertainty-based part of that space by extracting token probability, log-probability, and entropy features from Qwen models, training classical classifiers, and comparing performance across HaluEval with context, HaluEval without context, and TruthfulQA.

3 Datasets and Preprocessing

This project uses two hallucination and truthfulness datasets: HaluEval QA and TruthfulQA. HaluEval is a benchmark designed to evaluate whether LLM outputs are supported or hallucinated. TruthfulQA is a benchmark designed to test whether models reproduce common false beliefs or give truthful answers. Together, these datasets allow the project to compare context-grounded hallucination detection with no-context truthfulness detection.

HaluEval contains several task types, but only the QA subset is used in this project. This subset is appropriate for the main experiments because each example contains a question, a candidate answer, a supporting context passage, and a binary hallucination label. The processed HaluEval QA dataset contains 20,000 examples, with both supported and hallucinated answers.

TruthfulQA is used as a second evaluation dataset. It is different from HaluEval because it focuses on questions that are likely to trigger common misconceptions. For example, an incorrect answer may sound familiar or plausible because it matches a popular false belief. TruthfulQA does not provide a supporting context passage, so it represents a no-context truthfulness setting. In the processed version used here, correct answers are treated as truthful examples and incorrect answers are treated as hallucinated or false examples. The processed TruthfulQA dataset contains 5,918 examples.

Both datasets are converted into a common JSONL schema so that the same feature extraction and evaluation code can be used. Each processed record contains an identifier, dataset name, task name, prompt, context, answer, label, source field, and metadata. The most important fields for the experiments are the prompt, context, answer, and label. The label is defined consistently across datasets: label 0 means supported or truthful, while label 1 means hallucinated, unsupported, or incorrect.

The following examples show the difference between the two datasets. In HaluEval QA, the label is judged relative to a provided context passage:

> **HaluEval QA example. Question:** Who is the owner of a small publishing firm focused on a series of related science fiction role-playing games, the first published in 1977 by Game Designers' Workshop? **Context:** Roger Sanger is the owner of Digest Group Publications, a small publishing firm focused on the role-playing game market and specifically on the game Traveller. **Candidate answer:** Roger Sanger owns a company that focuses on publishing science fiction games. **Label:** 1, hallucinated.

This answer is labeled hallucinated because the context identifies Roger Sanger as the owner of Digest Group Publications, but the candidate answer changes the specific role-playing game focus into a broader claim about publishing science fiction games.

In TruthfulQA, there is no supporting context. The label is based on whether the answer is truthful or reflects a false belief:

> **TruthfulQA example. Question:** In what country was Barack Obama born? **Context:** none. **Candidate answer:** Barack Obama was born in Kenya. **Label:** 1, incorrect.

This example illustrates why TruthfulQA is more difficult for uncertainty-based detection. The answer can be phrased fluently and may reflect a familiar misconception, but the dataset does not provide an evidence passage that directly contradicts it.

The preprocessing step also standardizes the answer format. For HaluEval QA, each original example produces one supported answer and one hallucinated answer. For TruthfulQA, the available correct and incorrect answers are converted into separate labeled examples. This creates a

common binary classification setup, even though the original datasets were designed for somewhat different evaluation purposes.

Because context is central to this project, a no-context version of HaluEval QA is also created. In this version, the context field is removed before uncertainty feature extraction, while the question, answer, and label are kept unchanged. This produces a controlled comparison between HaluEval with context and HaluEval without context. The purpose is to measure how much the supporting passage helps the feature extractor assign more informative token probabilities and entropy values.

Dataset	Context Used	Examples	Purpose
HaluEval QA	Yes	20,000	Main context-grounded evaluation
HaluEval QA no-context	No	20,000	Context ablation experiment
TruthfulQA	No	5,918	No-context truthfulness evaluation

Table 1: Processed datasets used in the experiments.

After preprocessing, uncertainty features are extracted from each processed dataset using Qwen models. These feature files are stored as CSV files and are used as the direct input to the classification models. The raw text fields are not used directly as classifier features; instead, they are used during feature extraction to obtain numerical uncertainty signals. Table 2 summarizes the dependent and independent variables used in the classification task.

Variable	Role	Meaning
label	Dependent	Target class: 0 supported, 1 hallucinated
answer_length_tokens	Independent	Number of answer tokens
mean_token_probability	Independent	Average token confidence
min_token_probability	Independent	Lowest token confidence
max_token_probability	Independent	Highest token confidence
mean_token_logprob	Independent	Average token log-probability
min_token_logprob	Independent	Lowest token log-probability
max_token_logprob	Independent	Highest token log-probability
sum_token_logprob	Independent	Total answer log-probability
negative_mean_logprob	Independent	Average uncertainty from log-probability
low_confidence_token_ratio	Independent	Share of low-confidence tokens
mean_token_entropy	Independent	Average token uncertainty
min_token_entropy	Independent	Lowest token uncertainty
max_token_entropy	Independent	Highest token uncertainty
sum_token_entropy	Independent	Total answer entropy
high_entropy_token_ratio	Independent	Share of high-entropy tokens

Table 2: Dependent and independent variables used by the classifiers.

The prompt, context, and answer text are therefore not independent variables in the final classifier. They are inputs to the Qwen feature extraction step, and the classifier receives only the numerical features listed in Table 2.

The original example identifiers are preserved in the processed files. These identifiers are later used for grouped train-test splitting so that answer variants from the same original question are not split across training and test sets.

4 Methodology

4.1 Uncertainty Feature Extraction

The main idea of the methodology is to use an LLM as an uncertainty feature extractor rather than as the final hallucination judge. For each processed example, the prompt, optional context, and candidate answer are formatted into a scoring input. The Qwen model is then used to score the answer tokens conditioned on the preceding text. In other words, the model estimates how likely each answer token is given the question and, when available, the context passage.

The model is not used to generate a new answer during this step. Instead, the existing candidate answer from the dataset is forced through the model token by token, and the probability of each observed answer token is recorded. This is important because the goal is to evaluate the uncertainty of the given answer, not to replace it with a new model-generated response. In the HaluEval with-context setting, the scoring input contains the context, question, and answer. In the no-context settings, the scoring input contains only the question and answer.

For an answer token y_t, the model produces a probability distribution over the vocabulary. The probability assigned to the observed answer token is written as:

$$p_t = P(y_t \mid x, y_{<t})$$

where x is the prompt and optional context, and $y_{<t}$ is the answer prefix before token t. A high value of p_t means that the model considered the token likely in that position. A low value means that the token was less expected. The log-probability is computed as:

$$\log(p_t)$$

Log-probability is useful because it is numerically stable and additive across tokens. Since log-probabilities are usually negative, lower values indicate weaker model confidence. This project also uses the negative mean log-probability as an uncertainty feature, where higher values indicate greater uncertainty.

Entropy is used to measure how spread out the model's next-token probability distribution is. For a token position t, entropy is computed as:

$$H_t = -\sum_i P(v_i \mid x, y_{<t}) \log P(v_i \mid x, y_{<t})$$

where v_i represents a possible vocabulary token. Low entropy means the model strongly prefers a small number of next tokens. High entropy means the model is less certain because probability mass is distributed across many alternatives. This project aggregates token-level probabilities, log-probabilities, and entropy values into answer-level features such as mean token probability, minimum token probability, mean token entropy, maximum token entropy, low-confidence token ratio, and high-entropy token ratio.

The ratio features are included because a small number of uncertain tokens may be more informative than the average confidence of the whole answer. The low-confidence token ratio measures the share of answer tokens with probability below 0.1, while the high-entropy token ratio measures the share of answer positions with entropy above 2.0. These thresholds provide simple summary indicators for whether an answer contains unusually uncertain token positions.

These features are intended to capture whether the answer looks confident or uncertain from the perspective of the feature extractor model. The classifier does not see the raw wording of the answer directly. It only receives the numerical uncertainty summary produced from the answer tokens.

4.2 Feature Extractor Models

Two open-source Qwen models are used for feature extraction: Qwen2.5-3B and Qwen2.5-0.5B. Qwen2.5-3B is treated as the main feature extractor because it is small enough to run locally but large enough to provide meaningful token-level probabilities. Qwen2.5-0.5B is used as a smaller comparison model. The purpose of using the smaller model is to test whether uncertainty features remain useful when the feature extractor has less model capacity.

Both Qwen models are used in the same way. They are not asked, "Is this answer hallucinated?" Instead, they score the likelihood of the candidate answer token by token. This distinction is important because the project does not rely on direct LLM self-evaluation. The Qwen models produce uncertainty signals, and the final decision is made by separate machine learning classifiers trained on those signals.

The comparison between Qwen2.5-3B and Qwen2.5-0.5B also helps evaluate the effect of model size. A larger feature extractor may assign more informative probabilities because it has stronger language understanding and factual knowledge. However, smaller models are cheaper and faster to run. Therefore, comparing both models provides a practical view of the trade-off between computational cost and uncertainty feature quality.

This design also improves reproducibility. Both feature extractor models are open-source and can be run locally, so the experiment does not depend on a closed API or hidden model version.

4.3 Classification Models

After feature extraction, hallucination detection is treated as a standard supervised binary classification task. The dependent variable is the binary label, and the independent variables are the extracted uncertainty features listed in Table 2. Four classifiers are used: Logistic Regression, Linear Support Vector Machine, RBF-kernel Support Vector Machine, and Random Forest.

Logistic Regression is used as a simple linear baseline. It estimates the probability that an example belongs to the hallucinated class based on a weighted combination of the uncertainty features. This model is useful because it is interpretable and shows whether a linear relationship between uncertainty features and hallucination labels is already strong.

The Linear Support Vector Machine is another linear classifier, but it learns a decision boundary by maximizing the margin between the two classes. It is included because it often performs well on tabular feature sets and can be more robust than Logistic Regression when the classes are separated by a clear boundary in feature space.

The RBF-kernel Support Vector Machine is included as a non-linear SVM. It can learn curved decision boundaries, which is useful when the uncertainty features do not separate the two classes linearly. This is especially relevant for TruthfulQA, where the PCA visualization suggests stronger class overlap.

Random Forest is also used as a non-linear classifier. It combines many decision trees and can model interactions between features, such as cases where high entropy and low token probability together indicate hallucination. This makes Random Forest useful for testing whether non-linear feature combinations improve performance compared with the linear models.

Before training Logistic Regression, Linear SVM, and RBF SVM, the numerical features are standardized so that features with larger numeric ranges do not dominate the learning process. Random Forest does not require the same scaling because tree-based models split features by thresholds. Using both linear and non-linear classifiers gives a balanced comparison between simple decision boundaries and more flexible classification models. If non-linear models perform better, it suggests that feature interactions or curved boundaries are useful.

5 Experimental Setup

The experiments answer three questions: whether uncertainty features separate supported and hallucinated answers within the same dataset, whether classifiers trained on HaluEval QA generalize to TruthfulQA, and how context and feature extractor size affect performance. The main evaluation uses a grouped 80/20 train-test split, where 80% of question groups are used for training and 20% are held out for testing. Grouping is based on the original question identifier, so answer variants from the same question do not appear in both training and test sets.

The grouped 80/20 procedure is applied separately to HaluEval QA, HaluEval QA without context, and TruthfulQA. This gives a direct same-dataset estimate for each setting. To reduce dependence on one split, grouped 5-fold cross-validation is also used. Each fold is used once for testing while the remaining folds are used for training, and results are summarized using mean and standard deviation.

External transfer experiments test dataset generalization by training on HaluEval QA and testing directly on TruthfulQA, with no re-training on TruthfulQA. A second transfer setting trains on HaluEval QA without context and tests on TruthfulQA to examine whether removing context makes the source dataset closer to the no-context target dataset. The context ablation compares HaluEval QA with context against the same examples scored without context, isolating the effect of supporting evidence on uncertainty features.

The model-size comparison repeats the same evaluations with Qwen2.5-3B and Qwen2.5-0.5B feature extractors. This checks whether a much smaller extractor still produces useful uncertainty signals. Feature ablation compares three groups: confidence/log-probability features, entropy features, and all features combined. This tests how much each feature group contributes to performance.

All split-based experiments use a fixed random seed to make the results reproducible. The evaluation scripts save both aggregate metric tables and per-example prediction files. The prediction files are useful for later error analysis because they show which examples were predicted correctly or incorrectly by each classifier.

The experimental settings are complementary. Same-dataset tests measure performance under matched train and test conditions, while transfer tests measure whether learned patterns survive dataset shift. Context ablation and model-size comparison then help analyze why performance changes across settings.

Experiment	Data Used	Purpose
Grouped 80/20	Each dataset separately	Main train-test evaluation
Grouped 5-fold CV	Each dataset separately	More stable performance estimate
External transfer	HaluEval to TruthfulQA	Test dataset generalization
Context ablation	HaluEval with/without context	Measure context effect
Model-size comparison	Qwen2.5-3B vs Qwen2.5-0.5B	Measure extractor-size effect
Feature ablation	Feature groups	Measure feature-group importance

Table 3: Experimental settings used in the project.

The reported metrics are accuracy, precision, recall, F1 score, ROC-AUC, and confusion-matrix counts. F1 is emphasized because both false positives and false negatives matter; ROC-AUC is included because it evaluates ranking quality using classifier scores rather than only hard predictions. Together, these metrics show both threshold-based classification quality and score-based separability between supported and hallucinated answers.

6 Results

6.1 Grouped 80/20 Evaluation

Table 4 summarizes the main grouped 80/20 results using the Qwen2.5-3B feature extractor. HaluEval QA with context gives the strongest same-dataset performance. The best F1 score is 0.9885 with Linear SVM, and all classifiers are very close to each other. This indicates that the extracted uncertainty features separate supported and hallucinated HaluEval QA answers very clearly when context is available.

TruthfulQA is much harder. The best F1 score is 0.7037 with RBF SVM, while Random Forest gives the highest ROC-AUC at 0.6894. This shows that the same type of uncertainty features are less separable on TruthfulQA, although the non-linear RBF SVM gives a modest F1 improvement over the linear models. The result is expected because TruthfulQA is a no-context misconception dataset rather than a context-grounded hallucination dataset.

The external transfer setting is weaker. Training on HaluEval QA and testing directly on TruthfulQA gives a best F1 score of 0.6311 with Random Forest, with ROC-AUC below 0.5. This suggests that the decision patterns learned from HaluEval QA do not transfer cleanly to TruthfulQA. The result should be interpreted as evidence of dataset shift, not as a failure of the classifiers alone.

Setting	Best F1 Model	Accuracy	F1	ROC-AUC
HaluEval QA	Linear SVM	0.9885	0.9885	0.9986
TruthfulQA	RBF SVM	0.6272	0.7037	0.6830
HaluEval QA $\rightarrow$ TruthfulQA	Random Forest	0.4902	0.6311	0.3985
HaluEval QA no-context	Random Forest	0.9278	0.9276	0.9699

Table 4: Grouped 80/20 and transfer results using Qwen2.5-3B features.

Figures 1 and 2 visualize the feature space using a two-dimensional PCA projection. These plots are interpretive only; the actual classifiers use the full uncertainty feature set. HaluEval QA shows clearer separation, while TruthfulQA shows more overlap.

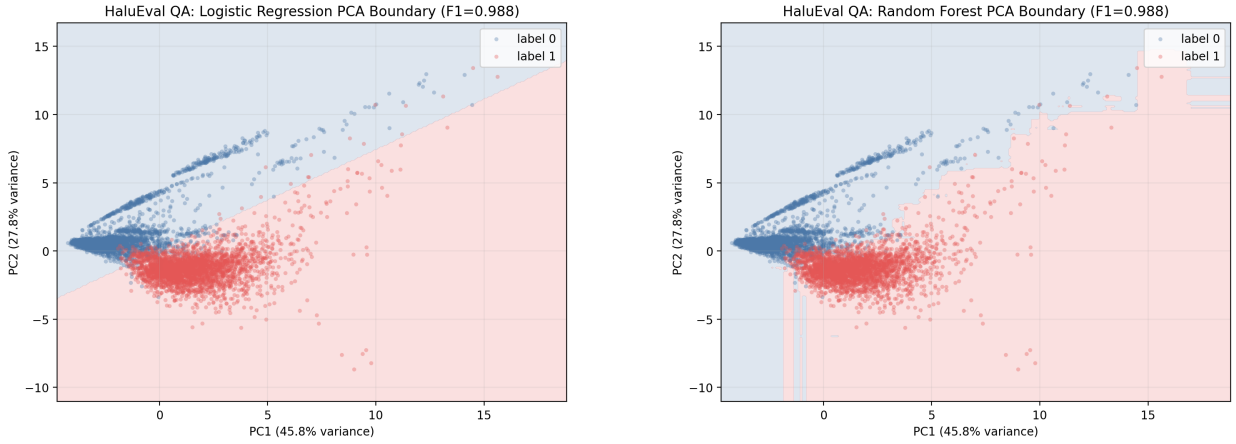

Figure 1: HaluEval QA PCA decision-boundary visualizations with a linear boundary and a Random Forest boundary. Both views show strong class separability.

Figure 2 compares a linear TruthfulQA boundary with the RBF SVM boundary. The RBF

SVM boundary is smoother and curved, which helps explain why it slightly improves TruthfulQA F1. However, the classes still overlap substantially, so the nonlinear model does not fully solve the TruthfulQA difficulty.

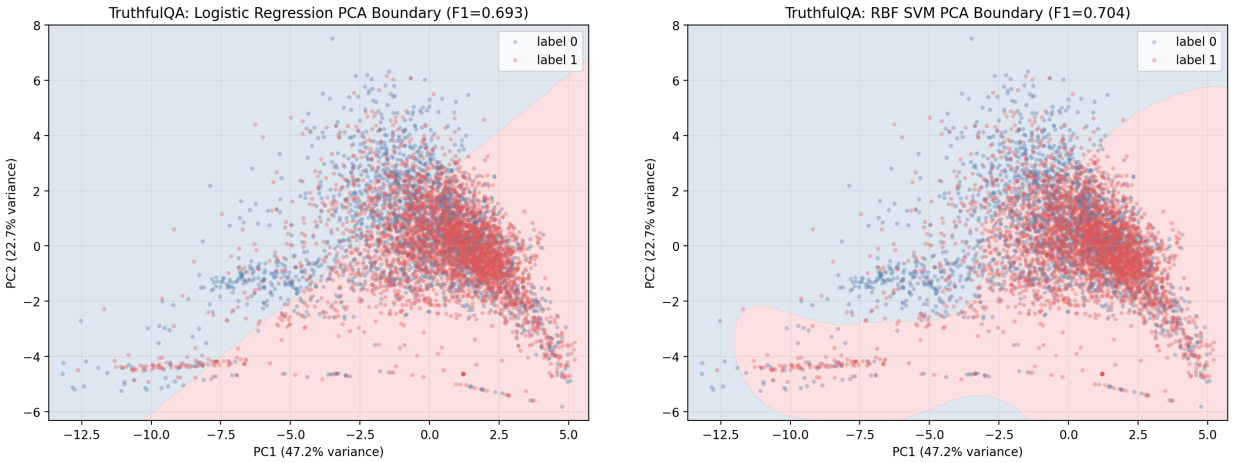

Figure 2: TruthfulQA PCA decision-boundary visualizations with a linear boundary and an RBF SVM boundary. The plot is interpretive only; final model training uses the full uncertainty feature set.

6.2 Grouped Cross-Validation

Grouped 5-fold cross-validation confirms the same overall pattern. HaluEval QA with context remains the easiest setting, with mean F1 around 0.988 for Qwen2.5-3B features. Qwen2.5-0.5B features also perform strongly on HaluEval QA, but with lower mean F1 around 0.976. This suggests that the smaller model still produces useful uncertainty features, although the larger model gives a small advantage in the context-grounded setting.

On TruthfulQA, both feature extractors perform much lower, with best mean F1 around 0.718 using RBF SVM. The difference between Qwen2.5-3B and Qwen2.5-0.5B is small in this setting. This suggests that TruthfulQA difficulty is not only caused by feature extractor size; it is also related to the no-context and misconception-focused nature of the dataset.

Dataset	Feature Extractor	Best F1 Model	Mean F1	Mean ROC-AUC
HaluEval QA	Qwen2.5-3B	RBF SVM	0.9883	0.9979
HaluEval QA	Qwen2.5-0.5B	RBF SVM	0.9759	0.9939
TruthfulQA	Qwen2.5-3B	RBF SVM	0.7173	0.6826
TruthfulQA	Qwen2.5-0.5B	RBF SVM	0.7181	0.6820

Table 5: Grouped 5-fold cross-validation summary for model-size comparison.

6.3 Context Ablation: HaluEval With and Without Context

The context ablation shows that context matters, but it does not fully explain the difference between HaluEval QA and TruthfulQA. With Qwen2.5-3B features, HaluEval QA cross-validation F1 drops from approximately 0.9883 with context to approximately 0.9277 without context. This is a meaningful decrease, showing that the supporting passage helps the feature extractor assign more useful uncertainty scores.

However, HaluEval QA without context still performs much better than TruthfulQA. This means that HaluEval no-context is not equivalent to TruthfulQA. HaluEval examples may still contain answer-style or question-answer artifacts that make the classification task easier. TruthfulQA is harder because many incorrect answers are plausible misconceptions, and there is no evidence passage to contradict them.

Setting	Feature Extractor	Best CV Model	Mean F1	Mean ROC-AUC
HaluEval with context	Qwen2.5-3B	RBF SVM	0.9883	0.9979
HaluEval no-context	Qwen2.5-3B	Random Forest	0.9277	0.9711
HaluEval no-context	Qwen2.5-0.5B	Random Forest	0.9271	0.9701
TruthfulQA no-context	Qwen2.5-3B	RBF SVM	0.7173	0.6826

Table 6: Context ablation and no-context comparison using grouped cross-validation.

6.4 ROC Curves

The ROC curves provide a score-based view of the same findings. HaluEval QA has near-perfect separability, and HaluEval QA without context remains strong. TruthfulQA has a much weaker ROC curve, showing that classifier scores do not rank truthful and incorrect examples as cleanly. This supports the conclusion that uncertainty features are highly effective for the context-grounded HaluEval QA setting but less reliable for no-context truthfulness detection.

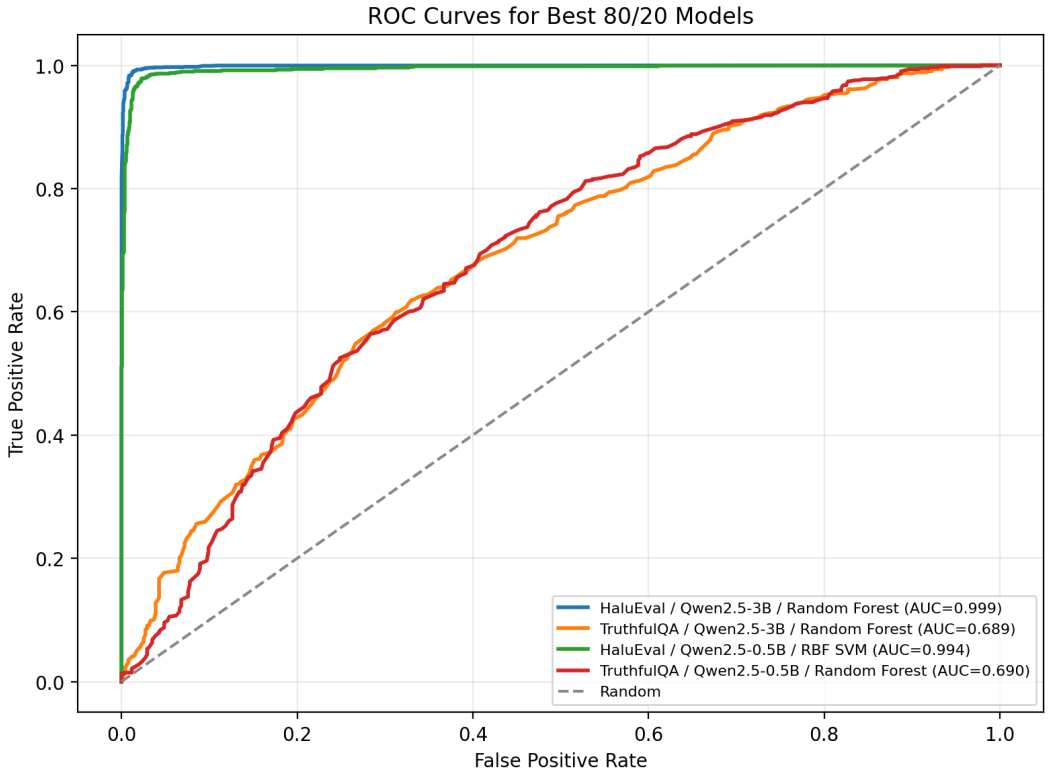

Figure 3: ROC curves for the best grouped 80/20 models.

7 Ablation Study

The ablation study tests whether hallucination detection depends mainly on confidence and log-probability features, entropy features, or the full feature set. This is important because the proposed method uses several related uncertainty signals, and ablation helps identify whether one group is carrying most of the predictive information.

Table 7 reports the main Qwen2.5-3B ablation results. On HaluEval QA, using all features gives the best F1 score, which suggests that confidence, log-probability, and entropy features provide complementary information in the context-grounded setting. Confidence and log-probability features alone are also strong, while entropy-only features are weaker but still informative.

TruthfulQA behaves differently. Entropy-only features with RBF SVM give the highest F1 in this grouped 80/20 split, while the full feature set gives stronger ROC-AUC than the entropy-only setting. This means that entropy is especially useful for TruthfulQA, but the overall separation between truthful and incorrect answers remains limited. Therefore, the ablation results support keeping the full feature set for the main detector while also showing that entropy is an important component.

Dataset	Feature Group	Best F1 Model	F1	ROC-AUC
HaluEval QA	Confidence + log-prob.	RBF SVM	0.9858	0.9980
HaluEval QA	Entropy	RBF SVM	0.9720	0.9921
HaluEval QA	All features	Linear SVM	0.9885	0.9986
TruthfulQA	Confidence + log-prob.	RBF SVM	0.6904	0.6359
TruthfulQA	Entropy	RBF SVM	0.7160	0.6491
TruthfulQA	All features	RBF SVM	0.7037	0.6830

Table 7: Feature-group ablation results using Qwen2.5-3B features and grouped 80/20 evaluation.

The full ablation outputs for Qwen2.5-0.5B and transfer evaluation are saved in the output tables. They show the same broad pattern: HaluEval benefits most from the full feature set, while TruthfulQA remains more sensitive to entropy and more difficult overall.

8 Error Analysis

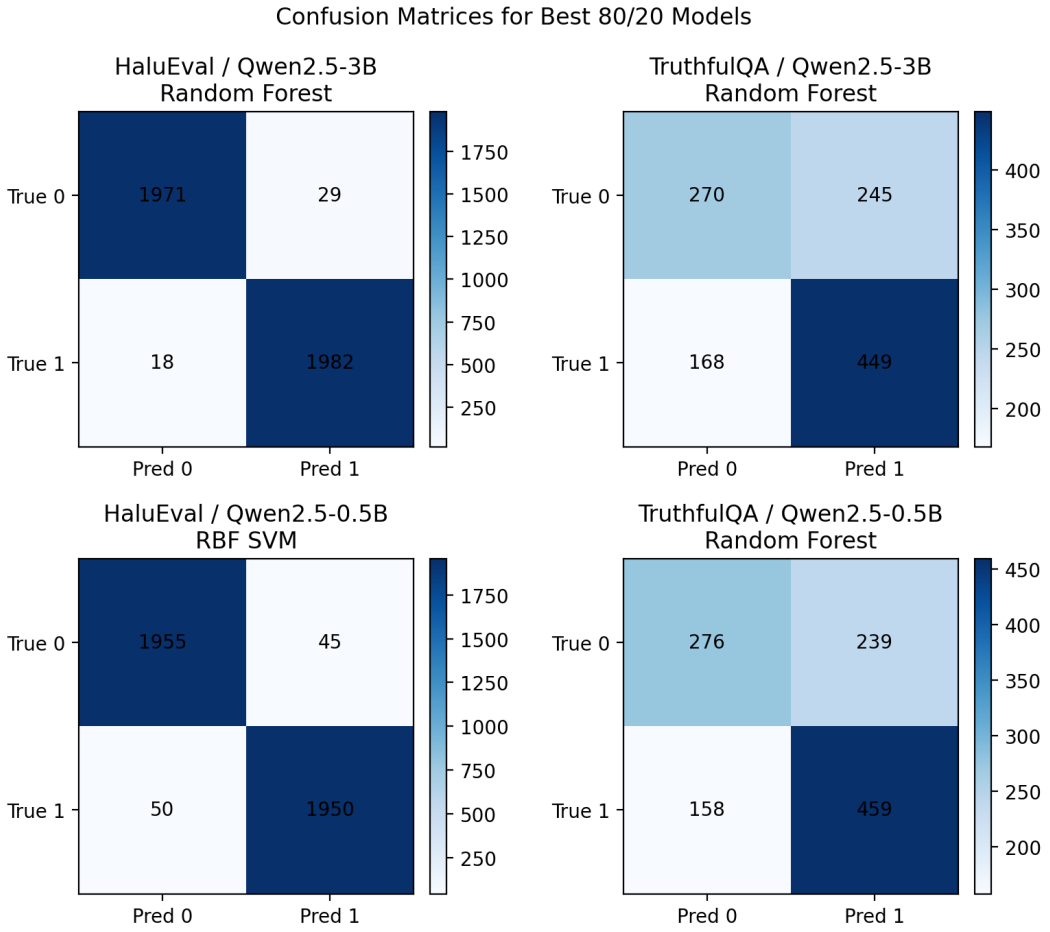

Figure 4: Confusion matrices for the best grouped 80/20 classifiers.

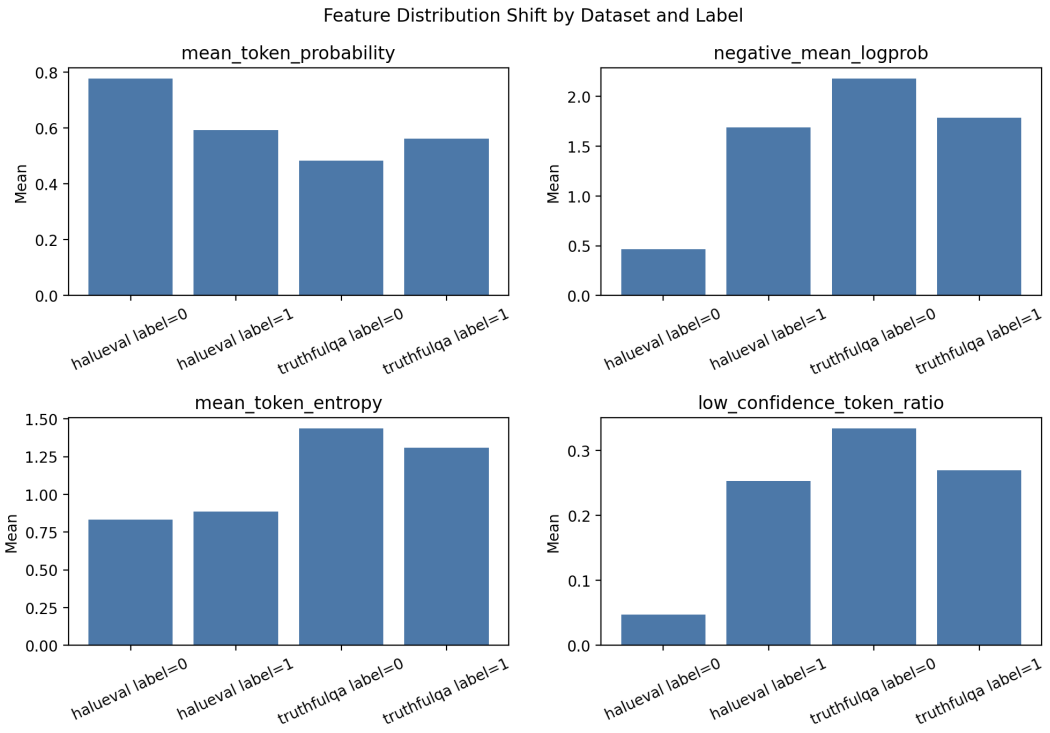

Figure 5: Selected uncertainty feature means by dataset and label.

9 Discussion and Limitations

10 Conclusion

References

[1] Artificial Intelligence and Machine Learning Review. Hallucination detection and confidence calibration for large language model outputs: Reproducible experiments on halueval. *Artificial Intelligence and Machine Learning Review*, 2025.

[2] Zhaorun Chen et al. A probabilistic framework for llm hallucination detection via belief tree propagation. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics*, 2025.

[3] Jinhao Duan et al. Enhancing uncertainty-based hallucination detection with stronger focus. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[4] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 2024.

[5] Robert Friel et al. Chainpoll: A high efficacy method for llm hallucination detection, 2023.

[6] Jannik Kossen et al. Semantic entropy probes: Robust and cheap hallucination detection in llms, 2024.

[7] Junyi Li et al. Halueval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[8] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.